(continued)

- Fewer bugs from the outset
- A comprehensive set of regression tests

Experienced developers will typically think about the design of their code before they do any coding, but expressing this design in the form of unit tests makes this process a lot more concrete. Before implementing any code, TDD practitioners imagine the code "they would like to have," which tends to result in cleaner, better-designed APIs. These unit tests also become examples of how to use the application code. And because at least one test is written for every new feature, the code tends to be better tested and have significantly fewer bugs.

Code developed using TDD also benefits from a comprehensive set of regression tests, which, coupled with the clean design that TDD encourages, makes the application easier to change and cheaper to maintain. This, along with the lower bug count, reduces maintenance costs and the total cost of ownership significantly.

On the downside, TDD isn't easy to learn, and it requires a lot of discipline, perseverance, and practice to adopt, especially without the guidance of an experienced TDD practitioner or the support of lead developers and management. As with any new technology or method, productivity will initially take a hit. Teams new to TDD will initially deliver more slowly, though the reduction in defects happens almost immediately.[1]

10.1.1 BDD is about writing specifications, not tests, at all levels

As you've seen, BDD involves discovering and specifying the behavior of a system, but this concept works just as well for the whole application as it does for an individual class: in both cases, you're specifying behavior.

Low-level BDD is a natural continuation of the BDD principles we've been applying to high-level requirements. In the chapters so far, you've seen how BDD practitioners express high-level requirements in the form of executable specifications. High-level requirements deal with the behavior of the system as a whole from the point of view of the business. Low-level requirements deal with the behavior of a component, class, or API from the point of view of the developer working with them. In both cases you're specifying behavior, and in both cases you can describe this behavior in terms of executable specifications. Only the target audience changes: high-level requirements are typically aimed at the broader team, whereas low-level, more technical requirements are aimed at future developers who will have to understand and maintain the application code.

[1] This seems to be supported by a case study from 2008, in which four teams (all new to TDD) reported 40–90% fewer defects, but observed a 15–35% increase in the initial development time. Nagappan, Maximilien, Bhat, and Williams, "Realizing quality improvement through test driven development: results and experiences of four industrial teams," *Empirical Software Engineering*, 13, no. 3 (June 2008): 289–302, http://dl.acm.org/citation.cfm?id=1380664.